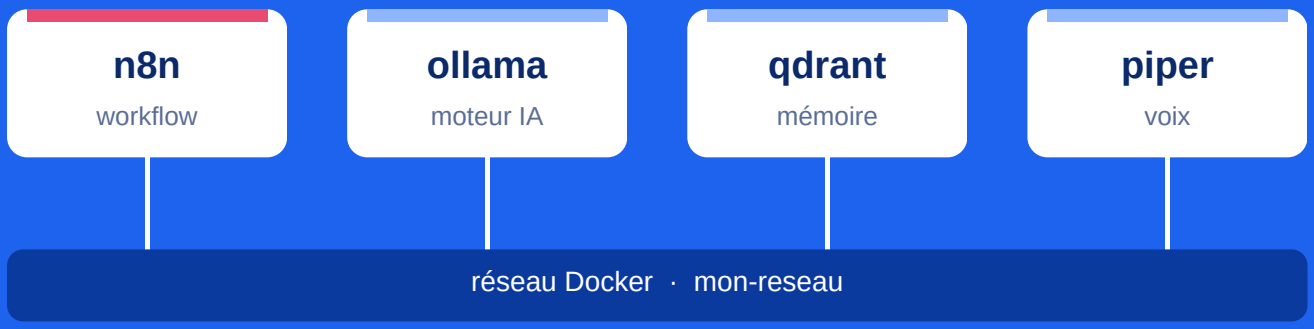
LABORATOIRE PERSONNEL

Laboratoire Docker et n8n

Agent IA local avec Ollama, Qdrant et Piper

Un agent avec mémoire et voix, 100 % en local



Docker · Images · Conteneurs · Volumes · Réseau Docker · n8n · Ollama · Llama 3.1 8B · Embeddings · Qdrant · RAG · Piper · API HTTP · macOS

Alexandre Peralta · Novembre 2026
Projet personnel · En formation Technicien PC & Réseaux

Résumé du projet

Construction d'un agent d'intelligence artificielle entièrement local sur un Mac, sans abonnement ni API payante. Quatre services tournent dans des conteneurs Docker reliés par un réseau dédié: n8n orchestre le traitement, Ollama fait tourner le modèle de langage Llama 3.1 8B et le modèle d'embeddings nomic-embed-text, Qdrant sert de mémoire vectorielle et Piper produit une voix française. L'agent reçoit une question dans un chat, retrouve les informations utiles dans sa mémoire, rédige une réponse fondée sur ce contexte puis la lit à voix haute. Chaque commande est construite à partir de la documentation officielle de l'image, et les difficultés rencontrées sont documentées avec leur résolution.

4 conteneurs Docker reliés par un réseau	**2** modèles d'IA en local	**28** figures documentées	**7** difficultés diagnostiquées et résolues

Compétences clés

Docker	**Docker Desktop**	**Volumes**	**Réseau Docker**
n8n	**Ollama**	**Llama 3.1 8B**	**Embeddings**
Qdrant	**RAG**	**Piper TTS**	**API HTTP**
Ligne de commande	**Conteneurs**	**Diagnostic**	**Documentation technique**

Pour aller plus loin dans ce document

- Architecture et environnement: section 2
- Installation des conteneurs (réseau, n8n, Ollama, Qdrant, Piper): sections 4 à 8
- Workflow n8n, réponse et mémoire: sections 9 et 10
- Test de l'agent et voix: sections 11 et 12
- Difficultés rencontrées: section 14
- Synthèse et compétences démontrées: section 16

Sommaire

1. Contexte et objectifs

2. Architecture et environnement

3. Point de départ: un Docker vide

4. Réseau Docker mon-reseau

5. n8n, le chef d'orchestre
- 5.1 Documentation officielle
- 5.2 Lancement du conteneur
- 5.3 Compte propriétaire

6. Ollama et les deux modèles
- 6.1 Téléchargement des modèles

7. Qdrant, la mémoire de l'agent

8. Piper, la voix de l'agent

9. Workflow n8n: répondre à partir de la mémoire
- 9.1 Connexions à Ollama et à Qdrant

10. Workflow n8n: remplir la mémoire
- 10.1 Vérification dans Qdrant

11. Test de l'agent

12. La voix avec Piper

13. Vue d'ensemble dans Docker Desktop

14. Difficultés rencontrées
- 14.1 Accent dans le nom du réseau
- 14.2 Image Piper qui exige une carte Nvidia
- 14.3 Port 5000 déjà occupé
- 14.4 JSON cassé par les guillemets courbes
- 14.5 Modèle de conversation dans le nœud Embeddings
- 14.6 Chaîne sans accès à la mémoire
- 14.7 Erreurs de saisie

15. Limites du laboratoire

16. Synthèse et compétences démontrées

1. Contexte et objectifs

Ce laboratoire construit un **agent d'intelligence artificielle entièrement local** sur un Mac, sans abonnement ni API payante. L'agent reçoit une question dans une fenêtre de chat, cherche les informations utiles dans sa propre mémoire, rédige une réponse avec un modèle de langage, puis lit cette réponse à voix haute.

Chaque brique tourne dans son propre conteneur **Docker**: **n8n** orchestre le traitement, **Ollama** fait tourner les modèles d'IA, **Qdrant** stocke la mémoire et **Piper** produit la voix. Les quatre conteneurs communiquent par un réseau Docker dédié.

Objectifs

- Installer et relier plusieurs services avec Docker en ligne de commande: images, conteneurs, volumes et réseau.
- Partir de la documentation officielle de chaque image pour écrire les commandes de lancement.
- Construire dans n8n une chaîne de type **RAG** (Retrieval-Augmented Generation): l'agent répond à partir de documents qu'on lui a donnés, et non de ce qu'il croit savoir.
- Ajouter une sortie vocale par une requête HTTP vers une API.

Les difficultés rencontrées pendant le laboratoire sont regroupées dans la section 14, avec leur diagnostic et leur résolution.

2. Architecture et environnement

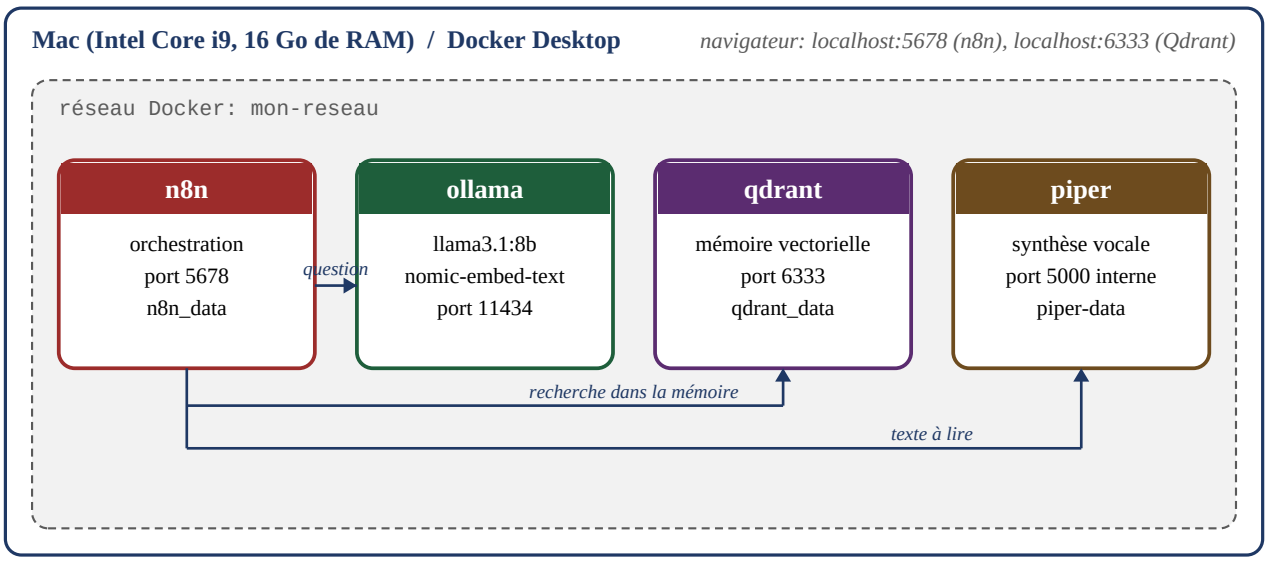


Figure 1. Architecture du laboratoire: quatre conteneurs reliés par le réseau mon-reseau.

Conteneur	Image	Port (Mac:conteneur)	Volume	Rôle
n8n	`n8nio/n8n`	5678:5678	n8n_data	Interface et workflows
ollama	`ollama/ollama`	11434:11434	ollama_data	Modèles de langage, en CPU
qdrant	`qdrant/qdrant`	6333:6333	qdrant_data	Base de données vectorielle
piper	`kamilkrawiec/piper-openai-tts`	5050:5000	piper-data	Synthèse vocale en français

Élément	Détail
Machine	MacBook Pro 16 pouces 2019, Intel Core i9, 16 Go de RAM, sans carte graphique Nvidia
Logiciel	Docker Desktop pour Mac, terminal zsh, navigateur Chrome
Modèles	llama3.1:8b (4,9 Go) pour répondre, nomic-embed-text (274 Mo) pour la mémoire

Sans carte graphique Nvidia, tous les modèles tournent sur le processeur. C'est pourquoi le choix s'est porté sur un modèle de 8 milliards de paramètres: un modèle de 70 milliards demanderait 40 à 50 Go de mémoire vive.

3. Point de départ: un Docker vide

Docker Desktop installe sur le Mac le moteur qui fait tourner les conteneurs et la commande `docker` du terminal. Il doit être ouvert avant de taper la moindre commande. Le laboratoire part d'une installation vide: aucun conteneur, volume ni image.

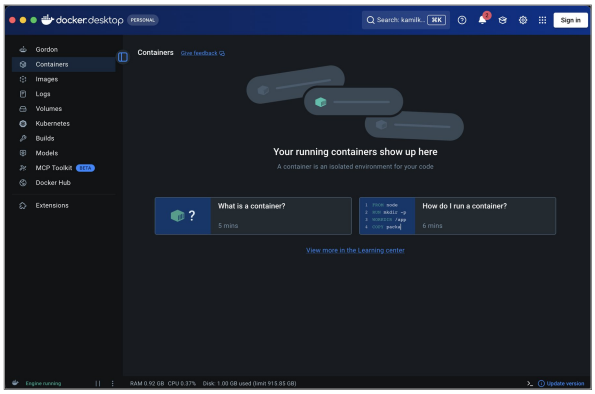


Figure 2. Docker Desktop: aucun conteneur.

```
alexandreperalta@macos-alexandre ~ % docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
alexandreperalta@macos-alexandre ~ % docker volume ls
DRIVER    VOLUME NAME
alexandreperalta@macos-alexandre ~ % docker images

IMAGE   ID             DISK USAGE   CONTENT SIZE   EXTRA
alexandreperalta@macos-alexandre ~ %
```

Figure 3. Aucun conteneur, volume ni image.

Image, conteneur et volume

- L'**image** est le modèle de départ téléchargé depuis Docker Hub. Elle ne change pas.
- Le **conteneur** est l'image en fonctionnement, l'équivalent du logiciel lancé. Il se supprime et se recrée en une commande.
- Le **volume** est le disque où le conteneur range ses données. Il est indépendant du conteneur: c'est le conteneur qui vient s'y brancher, ce qui permet de garder les données quand on recrée le conteneur.

4. Réseau Docker mon-reseau

Le réseau est créé **avant** les conteneurs. Chaque conteneur y est branché dès son lancement avec l'option `--network`, ce qui évite de les reconnecter un par un ensuite. Sur un réseau Docker personnalisé, les conteneurs se joignent par leur nom: n8n pourra appeler `http://ollama:11434` ou `http://piper:5000`.

```
alexandreperalta@macos-alexandre ~ % docker network create mon-reseau
8b8a9f81a993deeffd58dbf1d15e10ffb6decc0a769e2474e47284312d4c0847
alexandreperalta@macos-alexandre ~ %
```

Figure 4. Création du réseau: Docker renvoie son identifiant.

Le nom ne contient pas d'accent: le nom « mon-réseau », avec accent, provoque une erreur (section 14.1).

5. n8n, le chef d'orchestre

n8n est un outil d'automatisation où l'on construit un traitement en reliant des blocs, appelés **nœuds**. La version Community, gratuite, tourne en local sans limite d'utilisation.

5.1 Documentation officielle

Dans Docker Desktop, la barre de recherche en haut de la fenêtre permet de trouver une image (ici `n8nio/n8n`, publiée par un éditeur vérifié) et d'afficher sa documentation. Le site hub.docker.com donne la même page.

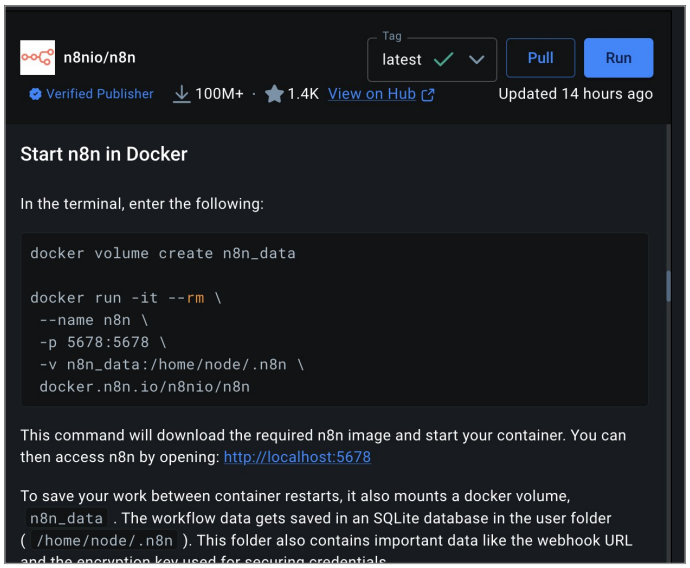


Figure 5. Documentation de n8nio/n8n dans Docker Desktop: port 5678 et volume /home/node/.n8n.

5.2 Lancement du conteneur

```
docker pull n8nio/n8n
```

```
alexandreperalta@macos-alexandre ~ % docker run -d --name n8n -p 5678:5678 -v n8n_data:/home/node/.n8n --network mon-reseau n8nio/n8n
2b67f572e4c8023b20409257f2d396bc1d4749745ae1bcf03d0be5b1d7db124d
alexandreperalta@macos-alexandre ~ %
```

Figure 6. Lancement de n8n en arrière-plan, sur le réseau mon-reseau, avec son volume.

Option	Rôle
`-d`	Lance le conteneur en arrière-plan et rend la main au terminal
`--name n8n`	Donne un nom fixe au conteneur, utilisé aussi comme nom sur le réseau
`-p 5678:5678`	Relie le port 5678 du Mac au port 5678 du conteneur
`-v n8n_data:/home/node/.n8n`	Range les workflows et les identifiants dans un volume nommé
`--network mon-reseau`	Branche le conteneur sur le réseau du laboratoire

La documentation utilise `--rm`, qui supprime le conteneur à son arrêt. Il a été retiré pour garder le conteneur et le relancer avec `docker start n8n`.

5.3 Compte propriétaire

À la première ouverture de `localhost:5678`, n8n demande de créer le compte propriétaire. Une adresse fictive suffit en local, car rien n'est envoyé.

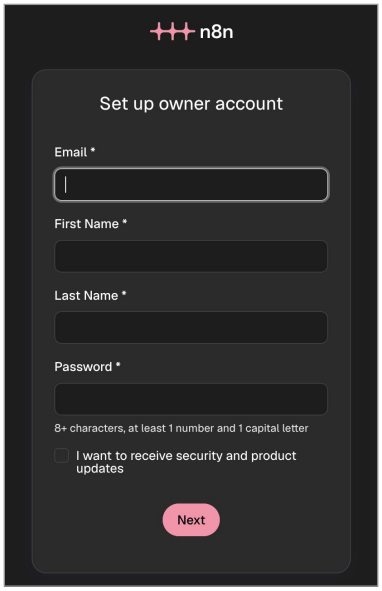


Figure 7. Création du compte propriétaire de n8n.

6. Ollama et les deux modèles

Ollama télécharge les modèles de langage, les charge en mémoire et répond aux demandes de n8n. Internet ne sert qu'au téléchargement des modèles: ensuite, tout fonctionne hors ligne.

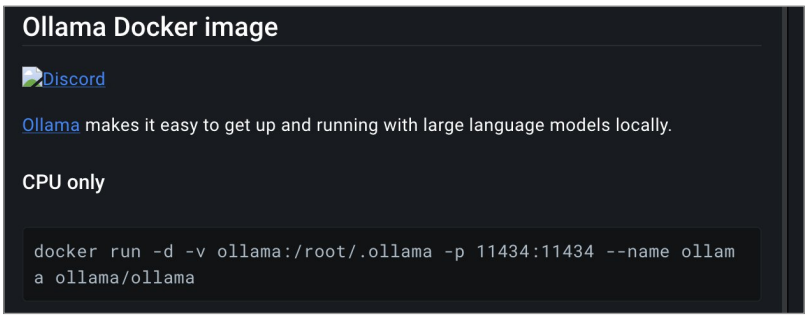


Figure 8. Documentation de ollama/ollama, partie CPU only: port 11434 et volume /root/.ollama.

La partie « Nvidia GPU » de la documentation ne concerne pas ce Mac. Le nom du volume est libre: la documentation l'appelle `ollama`, il est nommé ici `ollama_data`. Seul le chemin `/root/.ollama` dans le conteneur compte.

```
docker pull ollama/ollama
```

```
alexandreperalta@macos-alexandre ~ % docker run -d --name ollama -p 11434:11434 -v ollama_data:/root/.ollama --network mon-reseau ollama/ollama
cc8ca9f4eccb8f03a63599d29c382fa4449bca240d1f42f2d2156980e9c6349b
alexandreperalta@macos-alexandre ~ %
```

Figure 9. Lancement du conteneur Ollama.

6.1 Téléchargement des modèles

`docker exec` lance une commande à l'intérieur d'un conteneur qui tourne déjà. Le mot **ollama** apparaît deux fois: le premier est le nom du conteneur, le second est le programme qu'on exécute dedans.

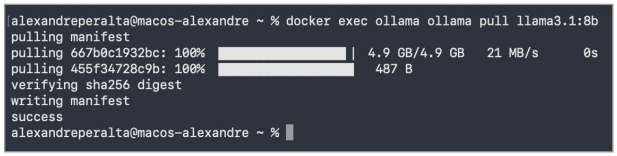


Figure 10. llama3.1:8b, le modèle qui répond.

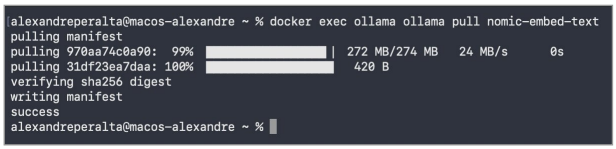


Figure 11. nomic-embed-text, le modèle de la mémoire.

Deux modèles sont nécessaires, car ils n'ont pas le même rôle. **llama3.1:8b** est un modèle de conversation: il rédige les réponses. **nomic-embed-text** calcule des **embeddings**: il traduit un texte en une suite de 768 nombres qui représente son sens. Deux textes de sens proche donnent des nombres proches, même si les mots diffèrent. C'est sur ces nombres que Qdrant fait ses recherches.

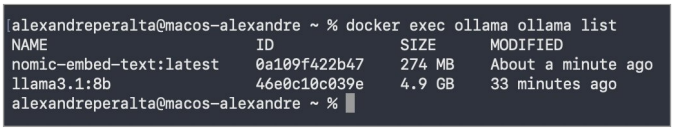


Figure 12. Les deux modèles installés dans le volume ollama_data.

7. Qdrant, la mémoire de l'agent

Qdrant est une base de données vectorielle: elle stocke les textes avec leurs embeddings et retrouve les passages les plus proches d'une question. Elle fournit aussi un tableau de bord web.

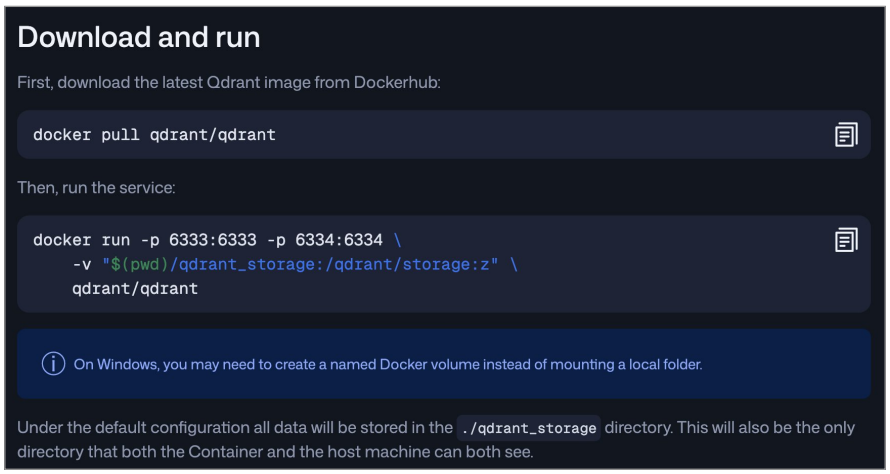


Figure 13. Documentation officielle de Qdrant (Local Quickstart).

La documentation range les données dans un dossier du Mac (`./qdrant_storage`). Un volume Docker nommé, `qdrant_data`, est utilisé à la place: il est géré par Docker et se supprime proprement. Le port 6334, ouvert dans la documentation, sert à une interface gRPC dont n8n n'a pas besoin: seul le port 6333 est publié.

```
docker pull qdrant/qdrant
```

```
alexandreperalta@macos-alexandre ~ % docker run -d --name qdrant -p 6333:6333 -v qdrant_data:/qdrant/storage --network mon-reseau qdrant/qdrant
bfad62e215ab735743635a431dcbda8758c0e39360ab80a472be2fcf54bf1d83
alexandreperalta@macos-alexandre ~ %
```

Figure 14. Lancement du conteneur Qdrant.

Le tableau de bord répond à l'adresse `localhost:6333/dashboard`. Il servira à vérifier la mémoire à la section 10.

8. Piper, la voix de l'agent

Piper est un moteur de synthèse vocale libre qui fonctionne sur processeur. L'image **kamilkrawiec/piper-openai-tts** l'expose sous forme d'API HTTP, avec la même route que l'API vocale d'OpenAI: `/v1/audio/speech`. Le choix de cette image vient de sa documentation sur GitHub, accessible depuis Docker Desktop par le lien du dépôt.

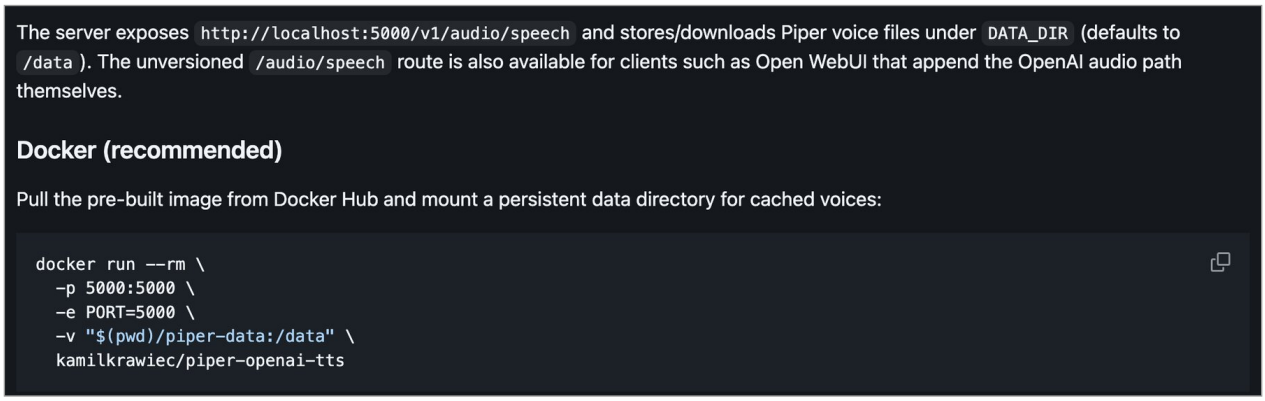


Figure 15. README du dépôt GitHub: route /v1/audio/speech, port 5000 et dossier /data.

Trois écarts avec la documentation: `--rm` est retiré pour garder le conteneur, un volume nommé remplace le dossier du Mac, et le port côté Mac devient **5050**, car le port 5000 du Mac est déjà occupé par le récepteur AirPlay de macOS.

```
docker pull kamilkrawiec/piper-openai-tts
```

```
alexandreperalta@macos-alexandre ~ % docker run -d --name piper -p 5050:5050 -v piper-data:/data --network mon-reseau kamilkrawiec/piper-openai-tts
46fc3d264c81a5060a40223b2c4083cf7fcc32c40d6e757ca87087431ceb01c2
alexandreperalta@macos-alexandre ~ %
```

Figure 16. Lancement du conteneur Piper.

> **Remarque sur le port de Piper**
>
> La capture montre `-p 5050:5050`, alors que le serveur Piper écoute sur le port 5000 dans le conteneur. La correspondance correcte pour joindre Piper depuis le Mac est `-p 5050:5000`. Le laboratoire n'en a pas souffert: n8n appelle Piper par le réseau mon-reseau, à l'adresse `http://piper:5000`, sans passer par le Mac. La publication du port n'est utile que pour tester Piper depuis le navigateur.

9. Workflow n8n: répondre à partir de la mémoire

Le workflow, nommé **Agent IA local**, comporte deux branches. La première, décrite ici, répond aux questions. La seconde (section 10) remplit la mémoire.

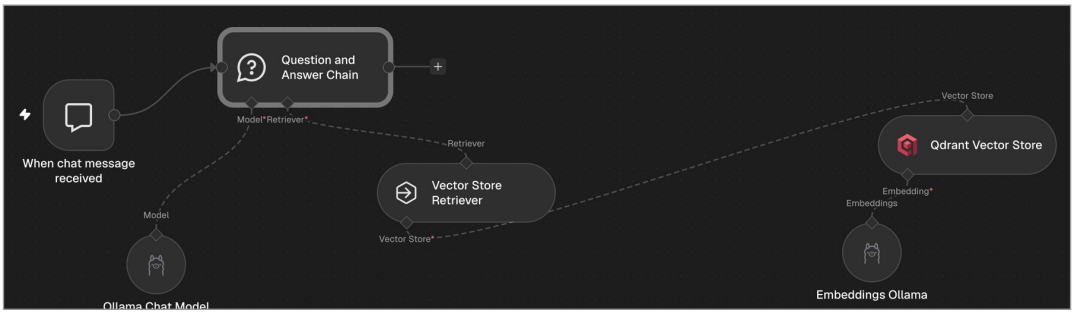


Figure 17. Branche de réponse: chat, Question and Answer Chain, modèle, retriever, Qdrant et embeddings.

Nœud	Rôle
When chat message received	Déclencheur: ouvre une fenêtre de chat et démarre le traitement à chaque message
Question and Answer Chain	Reçoit la question, demande les passages utiles au retriever, puis les transmet au modèle avec la question
Ollama Chat Model	Le modèle llama3.1:8b, qui rédige la réponse
Vector Store Retriever	Fait le lien entre la chaîne et la base de données vectorielle
Qdrant Vector Store	Cherche dans la collection memoire-agent, en mode Retrieve Documents
Embeddings Ollama	Calcule l'embedding de la question avec nomic-embed-text

La **Question and Answer Chain** est utilisée à la place de la Basic LLM Chain, qui n'a pas d'entrée Retriever et ne peut donc pas consulter la mémoire (section 14.6).

9.1 Connexions à Ollama et à Qdrant

Les identifiants de connexion (credentials) pointent vers les conteneurs par leur nom sur le réseau: `http://ollama:11434` pour Ollama et `http://qdrant:6333` pour Qdrant, sans clé API, puisque tout reste en local.

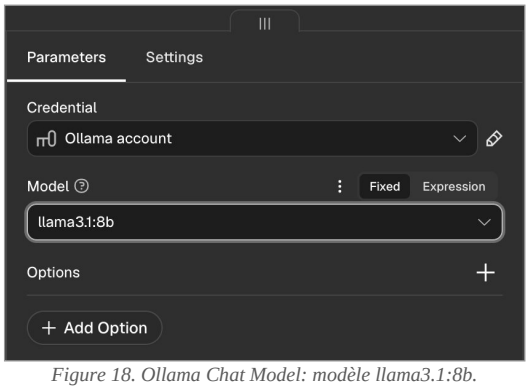


Figure 18. Ollama Chat Model: modèle llama3.1:8b.

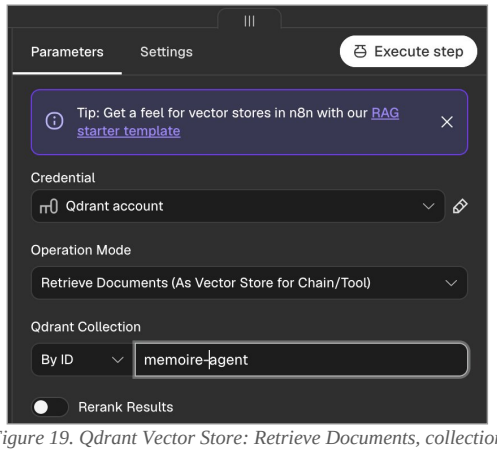


Figure 19. Qdrant Vector Store: Retrieve Documents, collection memoire-agent.

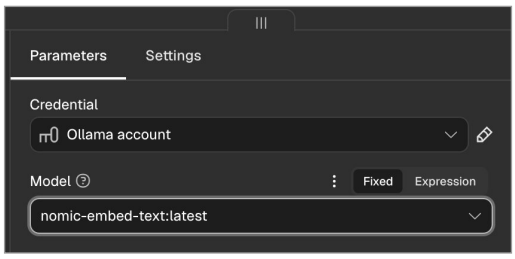


Figure 20. Embeddings Ollama: modèle nomic-embed-text.

Le point clé de cette branche est le choix du modèle dans le nœud Embeddings: un modèle de conversation comme llama3.1:8b ne sait pas calculer d'embeddings (section 14.5).

10. Workflow n8n: remplir la mémoire

La seconde branche écrit une information dans Qdrant. Elle démarre par un déclencheur manuel, **When clicking Execute workflow**, et n'est reliée à aucun nœud de la première branche.

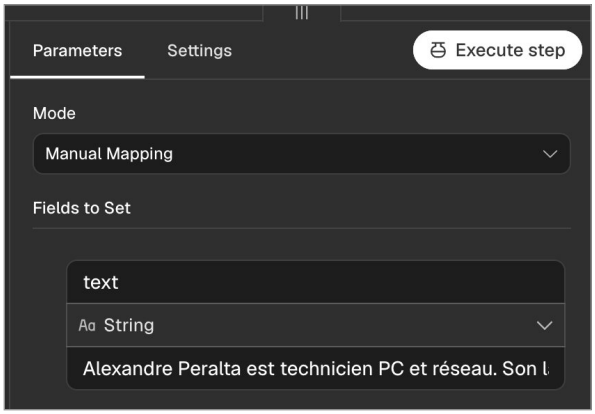


Figure 21. Edit Fields: le texte à mémoriser dans le champ text.

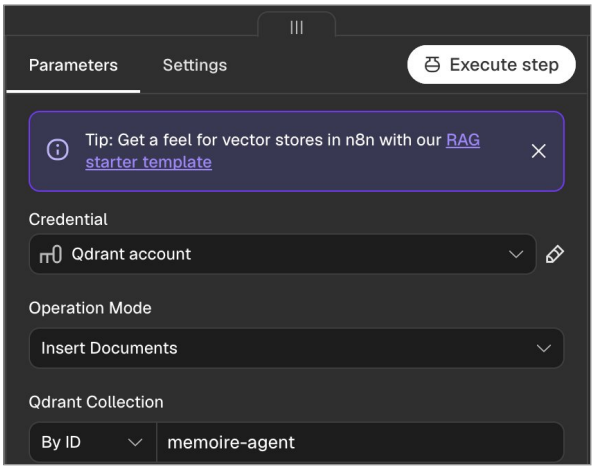


Figure 22. Qdrant Vector Store: mode Insert Documents.

Le texte choisi contient une information que le modèle ne peut pas connaître seul: « Alexandre Peralta est technicien PC et réseau. Son labo d'IA locale utilise n8n, Ollama, Qdrant et Piper. » Si l'agent la restitue, c'est qu'il l'a bien trouvée dans sa mémoire. Le nœud Qdrant utilise la même collection, **memoire-agent**, et son propre nœud Embeddings Ollama avec nomic-embed-text. Le **Default Data Loader** transforme les données reçues en document à indexer, avec ses réglages par défaut (JSON, Load All Input Data).

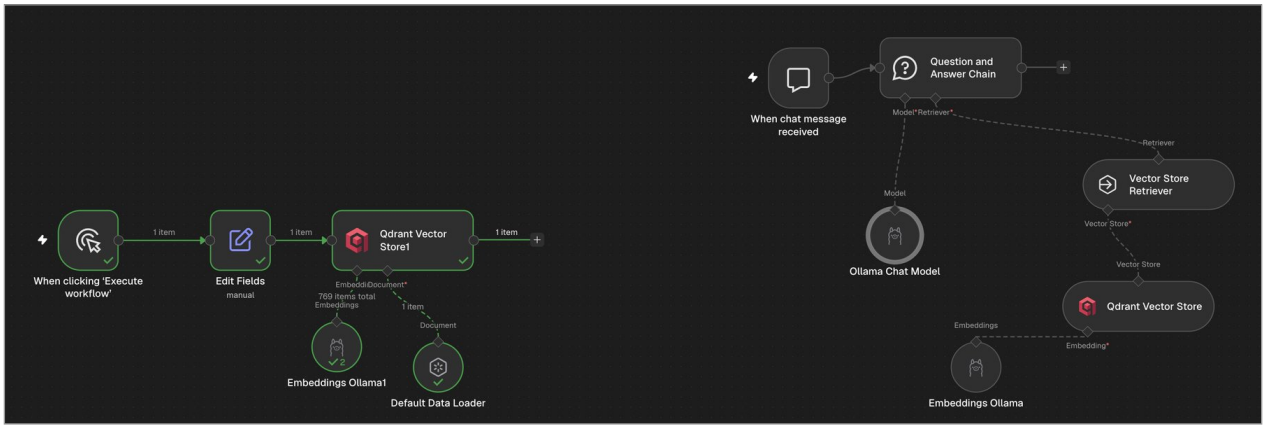


Figure 23. Exécution de la branche mémoire: tous les nœuds sont validés en 3,9 secondes.

La branche n'est exécutée qu'une fois: chaque exécution ajouterait une nouvelle copie du texte dans la collection.

10.1 Vérification dans Qdrant

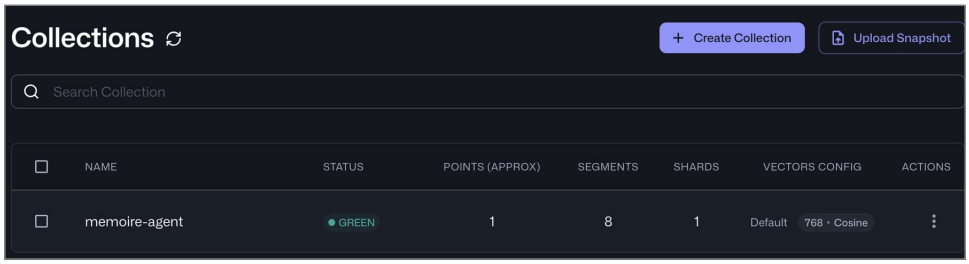


Figure 24. La collection memoire-agent contient 1 point, en vecteurs de taille 768.

Qdrant a créé la collection à la première écriture. Elle contient **1 point**, le texte enregistré, et ses vecteurs ont une taille de **768**, celle des embeddings de nomic-embed-text: c'est bien ce modèle qui a calculé l'embedding.

11. Test de l'agent

Dans la fenêtre de chat de n8n, la question posée est: « Qui est Alexandre Peralta? »

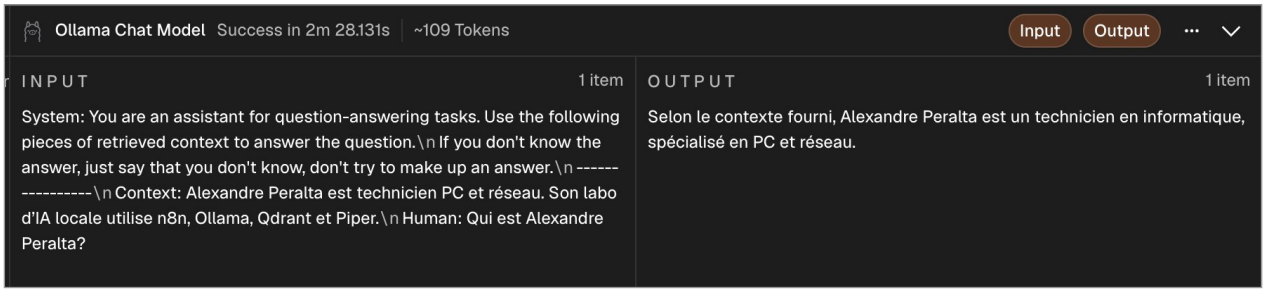


Figure 25. Entrée et sortie du modèle: le contexte retrouvé dans Qdrant, puis la réponse.

La partie **Input** montre ce que la chaîne envoie au modèle: une consigne qui lui demande de répondre à partir du contexte fourni, le **contexte** retrouvé dans Qdrant (le texte de la section 10), puis la question. La partie **Output** contient la réponse, fidèle au contexte: Alexandre Peralta est un technicien en informatique, spécialisé en PC et réseau.

La réponse a pris 2 minutes 28 secondes, car le modèle tourne uniquement sur le processeur.

12. La voix avec Piper

Un nœud **HTTP Request** est ajouté à la suite de la Question and Answer Chain. Il envoie la réponse du modèle à l'API de Piper, qui renvoie un fichier audio.

Paramètre	Valeur
Method	POST
URL	`http://piper:5000/v1/audio/speech`
Authentication	None
Body Content Type	JSON, en mode Using Fields Below
Champs du corps	model = piper, voice = fr_FR-siwis-medium, input = `{{ $json.response }}`

Le port est bien **5000**: n8n parle à Piper à l'intérieur du réseau Docker. Les champs sont saisis un par un plutôt qu'en JSON tapé à la main (section 14.4). Le champ **input** reçoit la réponse du modèle en faisant glisser le champ `response` depuis le panneau d'entrée.

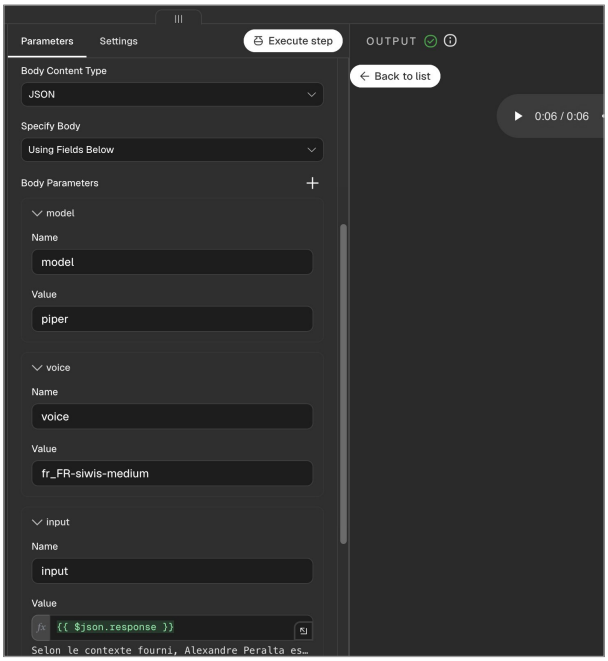


Figure 26. Nœud HTTP Request et fichier audio renvoyé par Piper.

Le fichier renvoyé est un WAV de 6 secondes, lu directement dans n8n: la voix française prononce la réponse de l'agent.

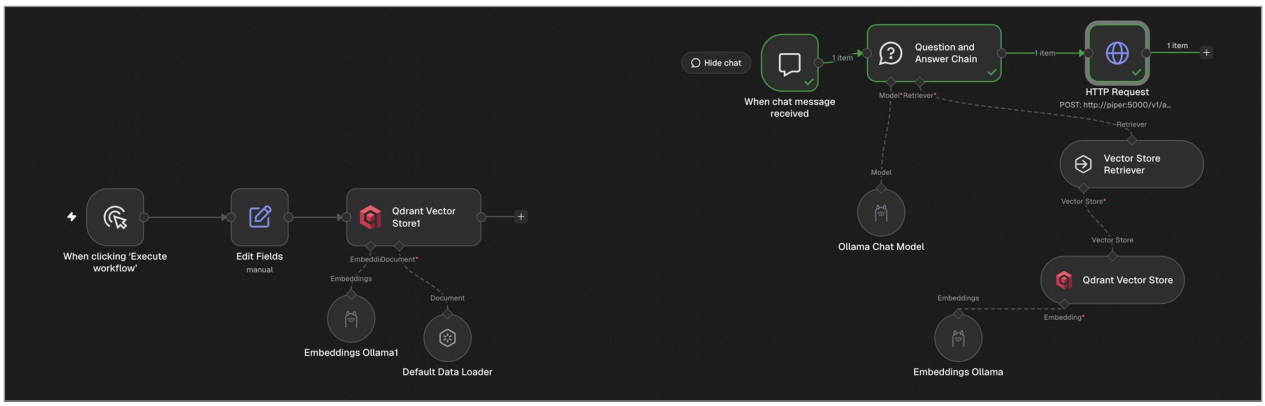


Figure 27. Workflow final: la branche de réponse se termine par le nœud HTTP Request vers Piper.

13. Vue d'ensemble dans Docker Desktop

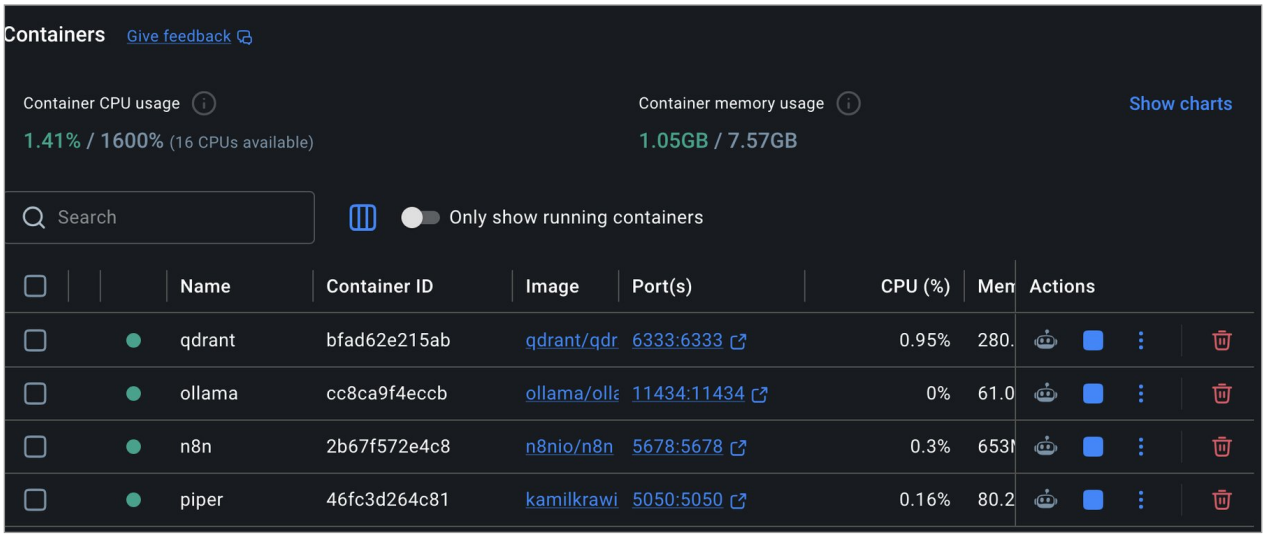


Figure 28. Les quatre conteneurs en fonctionnement dans Docker Desktop.

Les quatre conteneurs tournent ensemble et utilisent environ 1 Go de mémoire au repos. La ligne de Piper affiche le port 5050:5050 décrit dans la remarque de la section 8.

14. Difficultés rencontrées

Certaines solutions essayées pendant le laboratoire n'ont pas fonctionné. Chacune est présentée ici avec le constat, le diagnostic et le choix finalement retenu.

14.1 Accent dans le nom du réseau

Constat. Le réseau, d'abord nommé « mon-réseau », provoquait l'erreur `network not found` au lancement des conteneurs. **Résolution.** Réseau recréé sous le nom `mon-reseau`. Les noms Docker s'écrivent sans accent, en minuscules.

14.2 Image Piper qui exige une carte Nvidia

Constat. La première image Piper essayée ne démarrait pas sur ce Mac. **Diagnostic.** Elle s'appuie sur CUDA, donc sur une carte graphique Nvidia. **Résolution.** Image remplacée par `kamilkrawiec/piper-openai-tts`, qui fonctionne sur processeur.

14.3 Port 5000 déjà occupé

Constat. Le lancement de Piper sur le port 5000 du Mac échouait. **Diagnostic.** Ce port est utilisé par le récepteur AirPlay de macOS. **Résolution.** Publication sur le port 5050 côté Mac. Voir aussi la remarque de la section 8 sur la correspondance `5050:5000`.

14.4 JSON cassé par les guillemets courbes

Constat. Le corps JSON saisi à la main dans le nœud HTTP Request était refusé. **Diagnostic.** Le clavier du Mac remplaçait les guillemets droits par des guillemets typographiques, invalides en JSON. **Résolution.** Mode **Using Fields Below**: chaque champ est saisi séparément et n8n construit le JSON lui-même.

14.5 Modèle de conversation dans le nœud Embeddings

Constat. Au premier test, le nœud Embeddings Ollama renvoyait une erreur. **Diagnostic.** Il était réglé sur llama3.1:8b, un modèle de conversation. **Résolution.** Téléchargement de nomic-embed-text, modèle dédié aux embeddings, choisi dans les deux nœuds Embeddings.

14.6 Chaîne sans accès à la mémoire

Constat. La Basic LLM Chain, essayée en premier, ne permettait pas de relier Qdrant. **Diagnostic.** Cette chaîne n'a pas d'entrée Retriever. **Résolution.** Remplacement par la Question and Answer Chain.

14.7 Erreurs de saisie

Quelques fautes de frappe ont été repérées grâce aux messages de Docker: un conteneur créé sous le nom « olama », vu avec `docker ps -a` puis supprimé et recréé, ou un nom d'image écrit avec deux-points au lieu de la barre oblique (`unable to find image`). Dans n8n, un nœud désactivé par la touche D apparaissait avec la mention « Deactivated » et a été réactivé de la même façon.

15. Limites du laboratoire

- **Lenteur.** Sans carte graphique, une réponse prend environ deux minutes et demie.
- **Mémoire réduite.** La collection ne contient qu'un seul texte de test. Une vraie base de connaissances demanderait d'indexer des documents complets.
- **Voix dans n8n.** Le fichier audio est lu depuis l'interface de n8n, pas automatiquement sur les haut-parleurs. La reconnaissance vocale (parler à l'agent) n'est pas encore en place.
- **Sécurité.** Les services sont joignables sans authentification sur le Mac. C'est acceptable en local, pas sur un serveur exposé.
- **Lancement manuel.** Les conteneurs sont créés un par un en ligne de commande. Un fichier Docker Compose permettrait de tout décrire et de tout lancer en une seule commande.

16. Synthèse et compétences démontrées

Ce laboratoire met en place un agent d'IA complet, local et gratuit: quatre services dans des conteneurs Docker, reliés par un réseau dédié et pilotés par n8n. L'agent consulte sa mémoire avant de répondre, ce que prouvent le contexte transmis au modèle et la collection vérifiée dans Qdrant, puis lit sa réponse à voix haute. Les difficultés rencontrées sont documentées avec leur diagnostic et leur résolution.

Compétences démontrées

- Docker en ligne de commande: images, conteneurs, volumes nommés, réseau personnalisé, docker exec
- Lecture de documentation technique (Docker Hub, GitHub, sites officiels) et adaptation des commandes
- Ports et réseau: publication de ports, résolution par nom entre conteneurs, conflit de port sous macOS
- Automatisation avec n8n: déclencheurs, chaînes d'IA, credentials, requête HTTP vers une API
- IA locale: modèles de langage avec Ollama, embeddings, base vectorielle Qdrant, principe du RAG
- Diagnostic méthodique à partir des messages d'erreur
- Documentation technique d'un projet de bout en bout